3
Understanding React Components and Hooks

In this chapter, we will delve into the React components and their fundamental aspects and introduce you to the power of **Hooks**.

We will explore the essential concept of component data and how it shapes the structure of your React applications. We will discuss two primary types of component data: **properties** and **state**. Properties allow us to pass data to components, while state enables components to manage and update their internal data dynamically. We will examine how these concepts apply to function components and illustrate the mechanics of setting component state and passing properties.

In this chapter, we'll cover the following topics:

- Introduction to React components
- What are component properties?

- What is component state?
- React Hooks
- Maintaining state using Hooks
- Performing initialization and cleanup actions
- Sharing data using context Hooks
- Memoization with Hooks

Technical requirements

The code for this chapter can be found here: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter03**

Introduction to React components

React components are the building blocks of modern web and mobile applications. They encapsulate reusable sections of code that define the structure, behavior, and appearance of different parts of a user interface. By breaking down the UI into smaller, self-contained components, React enables developers to create scalable, maintainable, and interactive applications.

At its core, a React component is a JavaScript function or class that returns JSX syntax, which resembles HTML markup. In this book, we will focus mostly on function components, as

they have become the preferred approach for building components in recent years. Function components are simpler, more concise, and easier to understand compared to class components. They leverage the power of JavaScript functions and utilize React Hooks to manage state and perform side effects.

One of the primary advantages of using components in React is their reusability. Components can be reused across multiple parts of an application, reducing code duplication and increasing development efficiency. Moreover, components promote a modular approach to development, allowing developers to break down complex UIs into smaller, manageable pieces.

What are component properties?

In React, **component properties**, commonly known as **props**, allow us to pass data from a parent component to its child components. Props provide a way to customize and configure components, making them flexible and reusable. Props are read-only, meaning that the child component should not modify them directly. Instead, the parent component can update the props value and trigger a re-render of the child component with the updated data.

When defining a function component, you can access the props passed to it as a parameter:

```
const MyComponent = (props) => {
  return (
    <div>
      <h1>{props.title}</h1>
      <p>{props.description}</p>
    </div>
  );
};
```

In the above example, the `MyComponent` function component receives the props object as a parameter. We can access the individual props by using dot notation, such as `props.title` and `props.description`, to render the data within the component's JSX markup. It is also possible to access props using destructuring:

```
const MyComponent = ({ title, description }) => {
  return (
    <div>
      <h1>{title}</h1>
      <p>{description}</p>
    </div>
  );
};
```

As you can see, this approach is even cleaner and also allows us to use another destructuring feature, default values, which

we will discuss in this chapter.

Passing property values

React component properties are set by passing JSX attributes to the component when it is rendered. In *Chapter 7, Type Checking and Validation with TypeScript*, I'll go into more detail about how to validate the property values that are passed to components. Now let's create a couple of components in addition to `MyComponent` that expect different types of property values:

```
const MyButton = ({ disabled, text }) => {
  return <button disabled={disabled}>{text}</button>;
};
```

This simple button component expects a Boolean disabled property and a string text property. While we create components to show how we can pass the following props, you will notice how we already pass these properties to the button HTML element:

- **disabled property**: we put into the button attribute with the name `disabled`
- **text property**: we pass to the button as a child attribute

It's also important to know that any JavaScript expression you want to pass to the component should be wrapped with curly braces.

Let's create one more component that expects an array property value:

```
const MyList = ({ items }) => (
  <ul>
    {items.map((i) => (
      <li key={i}>{i}</li>
    ))}
  </ul>
);
```

You can pass just about anything you want as a property value via JSX, just as long as it's a valid JavaScript expression. The `MyList` component accepts an items property, an array that is mapped to `<li>` elements.

Now, let's write some code to set these property values:

```
import * as ReactDOM from "react-dom";
import MyButton from "./MyButton";
import MyList from "./MyList";
import MyComponent from "./MyComponent";
const root = ReactDOM.createRoot(document.getElementById("root"));
```

```
const appState = {
  text: "My Button",
  disabled: true,
  items: ["First", "Second", "Third"],
};
function render(props) {
  root.render(
    <main>
      <MyComponent
        title="Welcome to My App"
        description="This is a sample component."
      />
      <MyButton text={props.text} disabled={props.disabled} />
      <MyButton text="Another Button" disabled />
      <MyList items={props.items} />
    </main>
  );
}
render(appState);
setTimeout(() => {
  appState.disabled = false;
  appState.items.push("Fourth");
  render(appState);
}, 1000);
```

The `render` function looks like it's creating new React component instances every time it's called. React is smart enough to figure out that these components already exist, and that it only needs to figure out what the difference in output will be with

the new property values. In this example, the call to `setTimeout` causes a delay of 1 second. Then, the `appState.disabled` value is changed to false and the `appState.items` array has a new value added to the end of it. The call to `render` will re-render the components with new property values.

Another takeaway from this example is that you have an `appState` object that holds onto the state of the application. Pieces of this state are then passed into components as properties when the components are rendered. The state has to live somewhere and, in this case, it's outside of the component. We'll explore this approach in depth, and why it's important, in *Chapter 12, State Management in React*.

I hope you noticed we've rendered another button where we passed props in a different way:

```
<MyButton text="Another Button" disabled />
```

This is a valid JSX expression and in case we want to pass constant values to the components, we can pass strings without curly braces and pass the Boolean value `true`, just leaving the attribute name in the component.

Default property values

In addition to passing data, we can also specify default values for props using the `defaultProps` property. This is helpful when a prop is not provided, ensuring that the component still behaves correctly:

```
const MyButton = ({ disabled, text }) => (
  <button disabled={disabled}>{text}</button>
);
MyButton.defaultProps = {
  disabled: false,
  text: "My Button",
};
```

In this case, if the parent component does not provide the `text` or `disabled` props, the component will fall back to the default values specified in `defaultProps`.

As I mentioned before, with destructuring, we have a more convenient way to set up default props.

Let's take a look at the updated example of the `MyButton` component:

```
const MyButton = ({ disabled = false, text = "My Button" }) => (
  <button disabled={disabled}>{text}</button>
);
```

Using destructuring, we can define props and set the default value right inside the function. It's cleaner and easy to see in cases when we have a big component with a lot of props.

In the upcoming sections, we will further dive into component state with Hooks, and other key concepts.

What is component state?

In React, component state refers to the internal data held by a component. It represents the mutable values that can be used within the component and can be updated over time. State allows components to keep track of information that can change, such as user input, API responses, or any other data that needs to be dynamic and responsive.

State is a feature provided by React that enables components to manage and update their own data. It allows components to re-render when the state changes, ensuring that the user interface reflects the latest data.

To define state in a React component, you should use the `useState` hook inside of the component. You can then access and modify the state within the component's methods or JSX code. When the state is updated, React will automatically re-render the component and its child components to reflect the changes.

Before jumping to examples of using state in components, let's briefly explore what a React hook is.

React Hooks

React Hooks are a feature introduced in **React 16.8** that allows you to use state and other React features in functional components. Before Hooks, state management and lifecycle methods were primarily used in class components. Hooks provide a way to achieve similar functionality in functional components, making them more powerful and easier to write and understand.

Hooks are functions that enable you to "hook into" React's internal features, such as state management, context, effects, and more. They are prefixed with the `use` keyword (such as `useState`, `useEffect`, `useContext`, and so on). React provides several built-in Hooks, and you can also create custom Hooks to encapsulate reusable stateful logic.

The most commonly used built-in Hooks are:

- `useState`: This hook allows you to add state to a functional component. It returns an array with two elements: the current state value and a function to update the state.
- `useEffect`: This hook lets you perform side effects in your components, such as fetching data, subscribing to events, or

manually manipulating the DOM. It runs after every render by default and can be used to handle component lifecycle events like when the component is mounted, updated, or unmounted.

- `useContext`: This hook allows you to consume values from a React context. It provides a way to access context values without nesting multiple components.
- `useCallback` and `useMemo`: These Hooks are used for performance optimization. `useCallback` memoizes a function, preventing it from being recreated on every render, while `useMemo` memoizes a value, recomputing it only when its dependencies change.

We will examine all these Hooks in this chapter and will use them throughout the book. Let's continue with state and explore how we can manage it with the `useState` Hook.

Maintaining state using Hooks

The first React Hook API that we'll look at is called `useState`, which enables your functional React components to be stateful. In this section, you'll learn how to initialize state values and change the state of a component using Hooks.

Initial state values

When our components are first rendered, they probably expect some state values to be set. This is called the initial state of the component, and we can use the `useState` hook to set the initial state.

Let's take a look at an example:

```
export default function App() {
  const [name] = React.useState("Mike");
  const [age] = React.useState(32);
  return (
    <>
      <p>My name is {name}</p>
      <p>My age is {age}</p>
    </>
  );
}
```

The `App` component is a functional React component that returns JSX markup. But it's also now a stateful component, thanks to the `useState` hook. This example initializes two pieces of state, `name` and `age`. This is why there are two calls to `useState`, one for each state value.

You can have as many pieces of state in your component as you need. The best practice is to have one call to `useState` per state value. You could always define an object as the state of

your component using only one call to `useState`, but this complicates things because you have to access state values through an object instead of directly. Updating state values is also more complicated using this approach. When in doubt, use one `useState` hook per state value.

When we call `useState`, we get an array returned to us. The first value of this array is the state value itself. Since we've used array-destructuring syntax here, we can call the value whatever we want; in this case, it is `name` and `age`. Both of these constants have values when the component is first rendered because we passed the initial state values for each of them to `useState`. Here's what the page looks like when it's rendered:

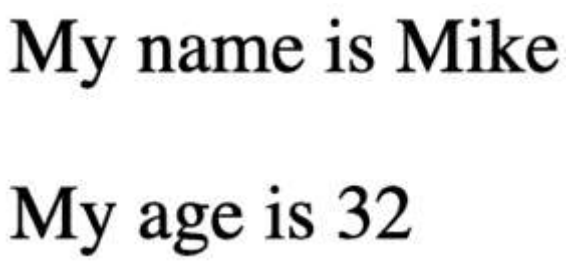

Figure 3.1: Rendered output using values from state Hooks

Now that you've seen how to set the initial state values of your components, let's learn about updating these values.

Updating state values

React components use state for values that change over time. The state values used by components start off in one state, as we saw in the previous section, and then change in response to some event: for example, the server responds to an API request with new data, or the user has clicked a button or changed a form field.

To update the state, the `useState` hook provides an individual function for every piece of state, which we can access from the returned array from the `useState` hook. The first item is the state value and the second is the function used to update the value. Let's take a look at an example:

```
function App() {
  const [name, setName] = React.useState("Mike");
  const [age, setAge] = React.useState(32);
  return (
    <>
      <section>
        <input value={name} onChange={(e) => setName(e.target.value)} />
        <p>My name is {name}</p>
      </section>
      <section>
        <input
          type="number"
          value={age}
```

```
            onChange={(e) => setAge(e.target.value)}
          />
          <p>My age is {age}</p>
        </section>
      </>
    );
  }
```

Just like the example from the initial state values section, the `App` component in this example has two pieces of state: `name` and `age`. Unlike the previous example, this component uses two functions to update each piece of state. These are returned from the call to `useState`. Let's take a closer look:

```
const [name, setName] = React.useState("Mike");
const [age, setAge] = React.useState(32);
```

Now, we have two functions: `setName` and `setAge`: that can be used to update the state of our component. Let's take a look at the text input field that updates the `name` state:

```
<section>
  <input value={name} onChange={(e) => setName(e.target.value)} />
  <p>My name is {name}</p>
</section>
```

Whenever the user changes the text in the `<input>` field, the `onChange` event is triggered. The handler for this event calls `setName`, passing it `e.target.value` as an argument. The argument passed to `setName` is the new state value of name. The succeeding paragraph shows that the text input is also updated with the new name value every time the user changes the text input.

Next, let's look at the `age` number input field and how this value is passed to `setAge`:

```
<section>
  <input
    type="number"
    value={age}
    onChange={(e) => setAge(e.target.value)}
  />
  <p>My age is {age}</p>
</section>
```

The `age` field follows the exact same pattern as the `name` field. The only difference is that we've made the input a number type. Any time the number changes, `setAge` is called with the updated value in response to the `onChange` event. The following paragraph shows that the number input is also updated with every change that is made to the `age` state.

Here is what the two inputs and their two corresponding paragraphs look like when they're rendered on the screen:

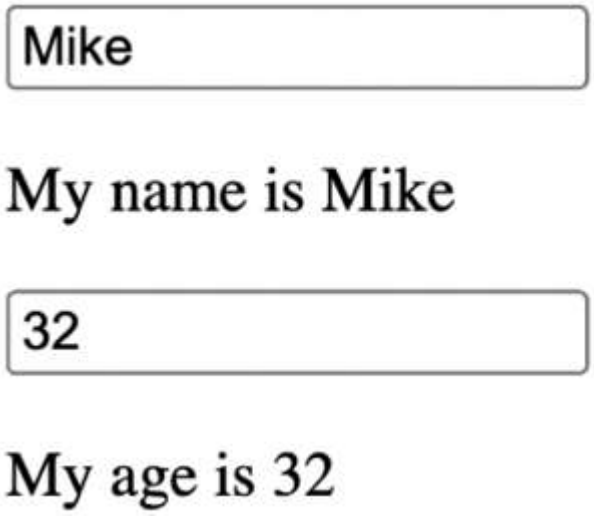

Figure 3.2: Using Hooks to change state values

In this section, you learned about the `useState` hook, which is used to add state to functional React components. Each piece of state uses its own hook and has its own value variable and its own setter function. This greatly simplifies accessing and updating state in your components. Any given state value should have an initial value so that the component can render correctly the first time. To re-render functional components that use state Hooks, you can use the setter functions that `useState` returns to update your state values as needed.

The next hook that you'll learn about is used to perform initialization and cleanup actions.

Performing initialization and cleanup actions

Often, our React components need to perform actions when the component is created. For example, a common initialization action is to fetch the API data that the component needs. Another common action is to make sure that any pending API requests are canceled when the component is removed. In this section, you'll learn about the `useEffect` hook and how it can help you with these two scenarios. You'll also learn how to make sure that the initialization code doesn't run too often.

Fetching component data

The `useEffect` hook is used to run "side effects" in your component. Another way to think about side-effect code is that functional components have only one job: returning JSX content to render. If the component needs to do something else, such as fetching API data, this should be done in a `useEffect` hook. For example, if you were to just make the API call as part of your component function, you would likely introduce race conditions and other difficult-to-fix buggy behavior.

Let's take a look at an example that fetches API data using Hooks:

```
function App() {
  const [id, setId] = React.useState("loading...");
  const [name, setName] = React.useState("loading...");
  const fetchUser = React.useCallback(() => {
    return new Promise((resolve) => {
      setTimeout(() => {
        resolve({ id: 1, name: "Mike" });
      }, 1000);
    });
  }, []);
  React.useEffect(() => {
    fetchUser().then((user) => {
      setId(user.id);
      setName(user.name);
    });
  });
  return (
    <>
      <p>ID: {id}</p>
      <p>Name: {name}</p>
    </>
  );
}
```

The `useEffect` hook expects a function as an argument. This function is called after the component finishes rendering, in a

safe way that doesn't interfere with anything else that React is doing with the component under the covers. Let's look at the pieces of this example more closely, starting with the mock API function:

```
const fetchUser = React.useCallback(() => {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve({ id: 1, name: "Mike" });
    }, 1000);
  });
}, []);
```

The `fetchUser` function is defined using the `useCallback` hook. This hook is used to memoize the function, meaning that it will only be created once and will not be recreated on subsequent renders unless the dependencies change. The `useCallback` accepts two arguments: the first is the function we want to memorize and the second is the list of dependencies that will be used to identify when React should re-create this function instead of using the memorized version. The `fetchUser` function is passed an empty array (`[]`) as the dependency list. This means that the function will only be created once during the initial render and won't be recreated on subsequent renders.

The `fetchUser` function returns a promise. The promise resolves a simple object with two properties, `id` and `name`. The `setTimeout` function delays the promise resolution for 1 second, so this function is asynchronous, just as a normal `fetch` call would be.

Next, let's look at the Hooks used by the `App` component:

```
const [id, setId] = React.useState("loading...");
const [name, setName] = React.useState("loading...");
React.useEffect(() => {
  fetchUser().then((user) => {
    setId(user.id);
    setName(user.name);
  });
});
```

As you can see, in addition to `useCallback`, we're using two Hooks in this component: `useState` and `useEffect`. Combining hook functionality like this is powerful and encouraged. First, we set up the `id` and `name` states of the component. Then, `useEffect` is used to set up a function that calls `fetchUser` and sets the state of our component when the promise resolves.

Here is what the `App` component looks like when it's first rendered, using the initial state of `id` and `name`:

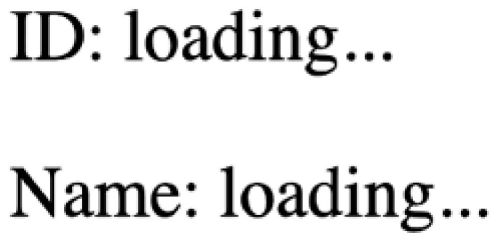

Figure 3.3: Displaying the loading text until the data arrives

After 1 second, the promise returned from `fetchUser` is resolved with data from the API, which is then used to update the ID and name states. This results in `App` being rerendered:

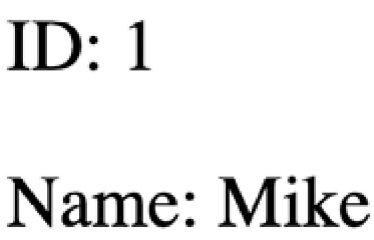

Figure 3.4: The state changes, removing the loading text and displaying returned values

There is a good chance that your users will navigate around your application while an API request is still pending. The `useEffect` hook can be used to deal with canceling these requests.

Canceling actions and resetting state

There's a good chance that, at some point, your users will navigate your app and cause components to unmount before responses to their API requests arrive. Sometimes your component can listen for some events and you should delete all listeners before unmounting the component to avoid memory leaks. In general, it's important to stop performing any background actions when a related component is deleted from the screen.

Thankfully, the `useEffect` hook has a mechanism to clean up effects such as pending `setInterval` when the component is removed. Let's take a look at an example of this in action:

```
import * as React from "react";
function Timer() {
  const [timer, setTimer] = React.useState(100);
  React.useEffect(() => {
    const interval = setInterval(() => {
      setTimer((prevTimer) => (prevTimer === 0 ? 0 : prevTimer - 1));
    }, 1000);
    return () => {
      clearInterval(interval);
    };
  }, []);
  return <p>Timer: {timer}</p>;
```

```
}
export default Timer;
```

This is a simple `Timer` component. It has the state `timer`, it sets up interval callback to update `timer` inside `useEffect()`, and it renders the output with the current `timer` value. Let's take a closer look at the `useEffect()` hook:

```
React.useEffect(() => {
  const interval = setInterval(() => {
    setTimer((prevTimer) => (prevTimer === 0 ? 0 : prevTimer - 1));
  }, 1000);
  return () => {
    clearInterval(interval);
  };
}, []);
```

This effect creates an interval timer by calling the `setInterval` function with a callback, which updates our `timer` state. The interesting thing you will notice here is that to the `setTimer` function, we are passing a callback instead of a number. It's a valid React API: when we need the previous state value to use to calculate a new one, we can pass a callback where the first argument is the current or 'previous' state value and we should return the new state value from this callback to update our state.

Inside `useEffect`, we are also returning a function, which React runs when the component is removed. In this example, the interval that is created by calling `setInterval` is cleared by calling the function that we returned from `useEffect`, where we call `clearInterval`. Functions that you return from `useEffect` will be triggered when the component is going to unmount.

Now, let's look at the `App` component, which renders and removes the `Timer` component:

```
const ShowHideTimer = ({ show }) => (show ? <Timer /> : null);
function App() {
  const [show, setShow] = React.useState(false);
  return (
    <>
      <button onClick={() => setShow(!show)}>
        {show ? "Hide Timer" : "Show Timer"}
      </button>
      <ShowHideTimer show={show} />
    </>
  );
}
```

The `App` component renders a button that is used to toggle the `show` state. This state value determines whether or not the `Timer` component is rendered, but by using the

`ShowHideTimer` convenience component. If show is true, `<Timer />` is rendered; otherwise, `Timer` is removed, triggering our `useEffect` cleanup behavior.

Here's what the screen looks like when it first loads:

Figure 3.5: A button used to initiate the state change

The `Timer` component isn't rendered because the `show` state of the `App` component is `false`. Try clicking on the show timer button. This will change the `show` state and render the `Timer` component:

Figure 3.6: Displays the timer

You can click on the **Hide Timer** button once more to remove the `Timer` component. Without the cleanup interval that we added to `useEffect`, this will create new listeners every time the timer is rendered, which will affect the memory leak.

React allows us to control when we want to run our effects. For example, when we want to all API requests after the first render, or we want to perform effects when a particular state changes. We'll take a look at how to do this next.

Optimizing side-effect actions

By default, React assumes that every effect that is run needs to be cleaned up and it should be run on every render. This typically isn't the case. For example, you might have specific property or state values that require cleanup and run one more time when they change. You can pass an array of values to watch as the second argument to `useEffect`: for example, if you have a resolved state that requires cleanup when it changes, you would write your effect code like this:

```
const [resolved, setResolved] = useState(false);
useEffect(() => {
  // ...the effect code...
  return () => {
    // ...the cleanup code
```

```
    };
  }, [resolved]);
```

In this code, the effect will be triggered and only ever run if the resolved state value changes. If the effect runs and the resolved state hasn't changed, then the cleanup code will not run and the original effect code will not run a second time. Another common case is never running the cleanup code, except for when the component is removed. In fact, this is what we want to happen in the example from the section on fetching user data. Right now, the effect runs after every render. This means that we're repeatedly fetching the user API data when all we really want is to fetch it once when the component is first mounted.

Let's make some modifications to the `App` component from the fetching component data requests example:

```
React.useEffect(() => {
  fetchUser().then((user) => {
    setId(user.id);
    setName(user.name);
  });
}, []);
```

We've added a second argument to `useEffect`, an empty array. This tells React that there are no values to watch and that we only want to run the effect once it is rendered and cleanup code when the component is removed. We've also added `console.count('fetching user')` to the `fetchUser` function. This makes it easier to look at the browser dev tools console and make sure that our component data is only fetched once. If you remove the `[]` argument that is passed to `useEffect`, you'll notice that `fetchUser` is called several times.

In this section, you learned about side effects in React components. Effects are an important concept, as they are the bridge between your React components and the outside world. One of the most common use cases for effects is to fetch data that the component needs, when it is first created, and then clean up after the component when it is removed.

Now, we're going to look at another way to share data with React components: context.

Sharing data using context Hooks

React applications often have a few pieces of data that are global in nature. This means that several components, possibly every component in an app, share this data: for example, information about the currently logged-in user might be used in several places. This is where the **Context API** comes in handy.

The Context API provides a way to create a shared data store that can be accessed by any component in the tree, regardless of its depth.

To utilize the Context API, we need to create a context using the `createContext` function from the **React** library:

```
import { createContext } from 'react';
const MyContext = createContext();
```

In the example above, we create a context called `MyContext` using `createContext`. This creates a context object that contains a `Provider` and a `Consumer`.

The `Provider` component is responsible for providing the shared data to its child components. We wrap the relevant portion of the component tree with the `Provider` and pass the data using the `value` prop:

```
<MyContext.Provider value={/* shared data */}>
  {/* Child components */}
</MyContext.Provider>
```

Any component within the `MyContext.Provider` can access the shared data using the `Consumer` component or the `useC-`

`ontext` hook. Let's take a look at how to read context using a hook:

```
import React, { useContext } from 'react';
const MyComponent = () => {
  const value = useContext(MyContext);
  // Render using the shared data
};
```

By utilizing the Context API, we can avoid the prop-drilling problem where data needs to be passed through multiple levels of components. It simplifies the process of sharing data and allows components to access the shared data directly, making the code more readable and maintainable.

It's worth noting that the Context API is not intended for all scenarios and should be used judiciously. It is most useful for sharing data that is truly global or relevant to a large portion of the component tree. For smaller-scale data sharing, props are still the recommended approach.

Memoization with Hooks

In React, function components are called on every render, which means that expensive computations and function creations can negatively impact performance. To optimize performance and prevent unnecessary recalculations, React provides

three Hooks: `useMemo`, `useCallback`, and `useRef`. These Hooks allow us to memoize values, functions, and references, respectively.

useMemo Hook

The `useMemo` hook is used to memoize the result of a computation, ensuring that it is only recomputed when the dependencies have changed. It takes a function and an array of dependencies and returns the memoized value.

Here's an example of using the `useMemo` hook:

```
import { useMemo } from 'react';
const Component = () => {
  const expensiveResult = useMemo(() => {
    // Expensive computation
    return computeExpensiveValue(dependency);
  }, [dependency]);
  return <div>{expensiveResult}</div>;
};
```

In this example, the `expensiveResult` value is memoized using `useMemo`. The computation inside the function is only executed when the `dependency` value changes. If the `dependency` remains the same, the previously memoized value is returned instead of recomputing the result.

useCallback hook

We already explored `useCallback` hook in this chapter, but I want to highlight one important use case. When a function component renders, all of its functions are recreated, including any inline callbacks defined within the component. This can lead to unnecessary re-renders of child components that receive these callbacks as props, as they perceive the callback as a new reference and trigger re-renders. Let's take a look at the example:

```
const MyComponent = () => {
  return <MyButton onClick={() => console.log("click")} />;
};
```

In this example, the inline function we provide to the `onClick` prop will be created every time `MyComponent` renders. It means the `MyButton` component will receive a new function reference each time, and as we already know, it will result in a new render for the `MyButton` component.

Here's an example that demonstrates the use of the `useCallback` hook:

```
const MyComponent = () => {
  const clickHandler = React.useCallback(() => {
```

```
    console.log("click");
  }, []);
  return <MyButton onClick={clickHandler} />;
};
```

In this example, the `clickHandler` function is memoized using `useCallback`. The empty dependency array `[]` indicates that the function has no dependencies and should remain constant throughout the component's lifecycle.

As a result, the same function instance is provided to `MyButton` on each render of `MyComponent`, preventing unnecessary re-renders of the child.

useRef hook

The `useRef` hook allows us to create a mutable reference that persists across component renders. It is commonly used to store values or references that need to be preserved between renders without triggering re-renders. Additionally, `useRef` can be used to access the DOM node or a React component instance:

```
const Component = () => {
  const inputRef = useRef();
  const handleButtonClick = () => {
    inputRef.current.focus();
```

```
  };
  return (
    <div>
      <input type="text" ref={inputRef} />
      <button onClick={handleButtonClick}>Focus Input</button>
    </div>
  );
};
```

In this example, the `inputRef` is created using `useRef`, and it is assigned to the `ref` attribute of the `input` element. This allows us to access the DOM node using the `inputRef.current` property. In the `handleButtonClick` function, we call the focus method on the `inputRef.current` to focus the input element when the button is clicked.

By using `useRef` to access the DOM node, we can interact with the underlying DOM elements directly without triggering re-renders of the component.

By leveraging memoization with the `useMemo`, `useCallback`, and `useRef` Hooks, we can optimize the performance of our React applications by avoiding unnecessary computations, preventing unnecessary re-renders, and preserving values and references across renders. This results in a smoother user experience and more efficient use of resources.

Summary

This chapter introduced you to React components and React Hooks. You learned about component properties or props by implementing code that passed property values from JSX to the component. Next, you found out what state is and how to manipulate it with the `useState` hook. Then, you learned about `useEffect`, which enables lifecycle management in functional React components, such as fetching API data when a component is mounted and cleaning up any pending async operations when it is removed. Then, you learned how to use the `useContext()` hook in order to access global application data. Lastly, you learned about memoization with the `useMemo`, `useCallback`, and `useMemo` Hooks.

In the following chapter, you'll learn about event handling with React components.